# Lecture 17 : Project Estimation Techniques (Contd.)

### Introduction

In the dynamic world of software development, accurately estimating project parameters like effort, cost, and duration is a cornerstone of successful project planning. Without reliable estimates, projects can suffer from budget overruns, missed deadlines, and dissatisfied stakeholders. This lecture delves deeper into various estimation techniques, building upon previous discussions, to equip aspiring project managers and developers with the tools to navigate this complex landscape. We will explore a taxonomy of methods, scrutinize the challenges of size estimation using traditional metrics, and examine several expert judgment-based approaches that leverage human experience and collective wisdom to forecast project outcomes. Understanding these techniques is crucial for making informed decisions, managing expectations, and ultimately delivering software projects efficiently and effectively.

### Core Concepts

| Term | Definition | Significance |
|---|---|---|
| `Project Estimation` | The process of forecasting the effort, cost, and duration required to complete a software project. | Essential for project planning, resource allocation, budgeting, and setting realistic expectations for stakeholders. |
| `Price to Win` | An estimation strategy where the project cost is determined by what the customer is willing or able to spend, rather than the actual work required. | Can secure contracts but often leads to inaccurate cost reflection, potentially compromising project quality or customer satisfaction. |
| `Person-month` | A unit of effort representing the amount of work one individual can typically accomplish in one month. | A standard metric for measuring project effort, allowing for calculation of total work required regardless of team size. |

| Term | Definition | Significance |
| --- | --- | --- |
| **`Source Lines of Code (SLOC)`** | A metric for project size based on counting the number of lines of source code in a program. | Conceptually simple but suffers from numerous shortcomings, making it less favored for modern software estimation. |
| **`Function Point (FP)`** | A metric for project size that quantifies the functionality provided to the user, independent of the programming language or technology. | Favored over SLOC due to its focus on functionality and better correlation with actual effort and complexity. |
| **`Expert Judgment`** | An estimation technique relying on the knowledge and experience of individuals or groups to predict project parameters. | A fundamental approach, especially when detailed information is scarce, but can be prone to bias and inaccuracy without proper structure. |
| **`Analogy Estimation`** | A method where the cost or effort of a new project is estimated by comparing it to similar past projects in the same application domain. | Provides accuracy when historical data from comparable projects is available, but requires a systematically maintained cost database. |
| **`Weighted Average Estimates`** | An estimation technique that combines optimistic, pessimistic, and most likely estimates using a specific formula to produce a more robust forecast. | Reduces the risk of relying on a single estimate by incorporating a range of possibilities, often used in sensitivity analysis. |
| **`Delphi Estimation`** | A structured expert judgment technique that uses multiple rounds of anonymous estimation and feedback to | Mitigates individual biases and groupthink by ensuring anonymity and structured feedback, leading to more objective estimates. |

| Term | Definition | Significance |
| --- | --- | --- |
| | achieve consensus without direct expert interaction. | |

## Detailed Analysis

### 1. Taxonomy of Estimation Methods

Project estimation methods can be broadly categorized based on their approach and underlying principles. Understanding this taxonomy helps in selecting the most appropriate technique for a given project context.

- **Top-down or Bottom-up:**
- These approaches are often activity-based and analytical.
- Top-down estimation starts with an overall project estimate and then breaks it down into smaller components.
- Bottom-up estimation begins by estimating individual tasks and then aggregates them to form a total project estimate.
- **Parametric or Algorithmic Models:**
- These models use mathematical formulas and historical data to estimate project parameters.
- An example is Function Points, which we will discuss further.
- **Expert Judgment:**
- Relies on the experience and intuition of experts.
- Can sometimes be perceived as "just guessing" but is formalized through various techniques.
- **Analogy:**
- A case-based and comparative approach.
- Estimates are derived by comparing the current project to similar past projects.
- **Price to Win:**
- A market-driven approach where the project cost is dictated by the customer's budget.

### 2. Price to Win Estimation

The "Price to Win" method is a unique estimation strategy driven by external market factors rather than internal project requirements.

- **Definition:** The project cost is set at whatever the customer is willing or able to spend on it.
- **Advantages:**
- The primary advantage is a high probability of securing the contract, as the bid aligns directly with the customer's financial expectations.
- **Disadvantages:**
- The most significant drawback is that the estimated costs often do not accurately reflect the actual work required to complete the project.
- This can lead to two undesirable outcomes: either the customer does not receive the desired system due to underestimation of work, or the customer overpays because the work required is less than the allocated budget.
- This approach can be seen as unethical and unbusiness-like because it disconnects cost from actual value and effort.
- **When it's appropriate:** Despite its ethical concerns, "Price to Win" may be the only viable strategy when detailed information regarding the volume of work is severely lacking.
- **Ethical Alternative:** A more ethical approach involves agreeing on project cost based on an outline proposal, with development constrained by that cost. A detailed specification can then be negotiated, or an evolutionary development approach can be used.

### 3. Project Parameters and Size Estimation

For effective project planning, two primary parameters need estimation: effort (which translates to cost) and duration. However, directly estimating these from a problem description is challenging.

- **Indirect Measurement:** Effort and duration are often measured indirectly through project size. Size is considered a fundamental measure of work.
- **Effort Measurement:** Effort is typically measured in "person-months."
- **One person-month** is defined as the amount of effort an individual can typically put into a project within one month.
- **Project Size Definition:** Project size is a measure of the problem's complexity, expressed in terms of the effort and time required to develop the product.

- **Key Metrics for Size:** Two popular metrics are used to measure project size:
- **Source Lines of Code (SLOC):** Also known as Lines of Code (LOC).
- **Function Point (FP):** This metric is increasingly favored over SLOC due to the latter's numerous shortcomings.

**4. Source Lines of Code (SLOC) Analysis**

SLOC, while conceptually simple, presents significant challenges as a reliable estimation metric.

- **Conceptual Simplicity:** SLOC was initially proposed when programs were typed on punch cards, with one line per card, making counting straightforward.
- **Challenges in Modern Programming:**
- **Multi-line statements:** In languages like Java, a single statement can span several lines, or multiple statements can exist on one line, making a simple line count ambiguous.
- **What to count:** It's unclear whether to count code for Graphical User Interfaces (GUIs) or built-in classes provided by programming languages as part of the system's SLOC.
- **Evolution of Software Development:** Initially, software development was primarily about writing code. Modern development involves many other activities (analysis, design, testing) that SLOC doesn't account for.
- **Terminology:**
- **LOC:** Line of Code
- **KLOC:** Thousands of Lines of Code
- **KSLOC:** Thousands of Source Lines of Code
- **NCKSLOC:** New or Changed KSLOC (indicating new code or modifications)
- **Counter-intuitive Aspects:**
- **Language Level vs. Productivity:** The metric suggests that the lower the level of the language, the more productive the programmer, because the same functionality often requires more lines of code in a lower-level language. This is misleading, as higher-level languages enable more functionality per line.
- **Verbosity vs. Productivity:** SLOC implies that programmers who write more verbose code are more productive than those who write compact code, simply because their line count is higher. This incentivizes less efficient coding practices.

- **Major Shortcomings:**
- **Coding Style Variation:** Size can vary significantly based on individual coding styles.
- **Focus on Coding Only:** SLOC exclusively focuses on the coding activity, ignoring effort spent on requirements analysis, design, and testing.
- **Poor Correlation:** It correlates poorly with the quality and efficiency of the code.
- **Penalizes High-Level Languages and Reuse:** Using high-level languages or reusing existing code reduces the line count, which SLOC interprets as lower productivity or smaller size, despite potentially higher efficiency.
- **Difficult Early Estimation:** It's very difficult to estimate SLOC accurately at the beginning of a project from just a problem description, making it less useful for initial project planning.
- **Programmer-Dependent:** Different programmers will write the same functionality with varying lines of code.
- **Limited Complexity Measurement:** SLOC only measures lexical/textual complexity, failing to consider structural or logical complexity.
- **Further Difficulties:**
- **Ambiguity with Advancements:** Rapid changes in programming methodologies, languages, and tools (e.g., language advancements, automatic source code generation, custom software, code reuse, object-orientation) make SLOC an ambiguous and less suitable metric. For instance, automatically generated code inflates SLOC without reflecting actual human effort.

## 5. Expert Judgment-based Estimation Techniques

These techniques leverage human expertise to estimate project parameters, often when quantitative data is scarce or unreliable.

#### 5.1 Basic Expert Judgment

- **Process:** One or more experts predict software costs. This process is iterated until a consensus is reached among the experts.
- **Advantages:**
- Relatively simple to implement.
- Can be accurate if the experts possess direct experience with similar systems.

- **Disadvantages:**
- Highly inaccurate if suitable experts are not available for the specific type of system being developed (e.g., an avionics system without avionics experts).
- **Steps:**
- A coordinator presents each expert with project specifications and an estimation form.
- A group meeting is held for experts to discuss estimation issues with the coordinator and each other.
- Experts fill out forms anonymously.
- The coordinator prepares and distributes a summary of the estimations on an iteration form.
- Another group meeting is called, specifically to discuss points where estimates varied widely.
- Experts fill out forms anonymously again.
- Steps 4 and 6 are iterated until a consensus decision is reached.
- **Expert Profile:** An expert is familiar with and knowledgeable about the application area and the technologies involved. This method is particularly appropriate when modifying existing code.
- **Connection to Analogy:** Research suggests that expert judgment often implicitly relies on an analogy methodology.

#### 5.2 Analogy Estimation

- **Process:** The cost of a project is computed by comparing it to a similar project that has already been completed in the same application domain.
- **Stages:**
- Identify significant features of the current project.
- Look for previous projects with similar features.
- Identify differences between the current and previous projects.
- Determine possible reasons for errors or risks associated with these differences.
- Adopt measures to reduce uncertainty or risk.
- **Advantages:**
- Can be accurate if reliable project data from past projects is available, and if the people and tools involved are similar.
- **Disadvantages:**
- Impossible to use if no comparable project has been tackled previously.

- Requires a systematically maintained cost database of past projects for effective comparison.
- **Cons of Basic Expert Judgment (General):**
- Hard to quantify and document the factors used by experts.
- Experts can be biased (optimistic or pessimistic), even with group consensus.
- Expert judgment typically complements other methods, such as algorithmic approaches.

#### 5.3 Weighted Average Estimates

- **Also known as:** Sensitivity analysis estimating.
- **Approach:** Instead of a single estimate, three estimates are obtained:
- **O (Optimistic):** The best-case scenario.
- **P (Pessimistic):** The worst-case scenario.
- **M (Most Likely):** The median or most probable case.
- **Formula:** These three values are then used in the following formula to produce a more accurate estimated effort:

```
Estimated effort = (O + 4M + P) / 6
```

- **Significance:** This method provides a more robust estimate by considering a range of possibilities and weighting the most likely scenario more heavily.

#### 5.4 Consensus Estimating

- **Process:** A structured approach involving a team to arrive at a collective estimate.
- **Steps:**
- A briefing on the project is provided to the estimating team.
- Each person receives a list of work components to estimate.
- Each person independently estimates O, M, and P values for each work component.
- The estimates are written on a whiteboard for all to see.
- Each person discusses the basis and assumptions behind their estimates.
- A revised set of estimates is produced based on the discussion.
- Averages for the O, M, and P values are calculated.

- These averaged values are then used in the weighted average formula `(O + 4M + P) / 6`.

#### 5.5 Delphi Estimation

- **Nature:** A variation of the consensus estimation technique, emphasizing anonymity.
- **Team Structure:** Involves a team of experts and a coordinator.
- **Key Difference from Basic Expert Judgment:** Experts never meet each other to discuss their viewpoints directly. The coordinator plays a crucial role in facilitating communication.
- **Process (Rounds):**
- Experts carry out estimation independently, providing rationale and assumptions.
- The coordinator notes any extraordinary rationale and circulates the estimations and rationales among the other experts.
- After receiving feedback from other experts, each expert re-estimates their values.
- This process is conducted in "rounds" (two or more). Starting from the second round, feedback from previous rounds is provided.
- Experts reassess their estimates, influenced by the opinions of others, but maintaining anonymity.
- **Importance:** Anonymity is crucial in Delphi, as it helps mitigate individual biases and prevents dominant personalities from unduly influencing the group's estimates.
- **Steps (similar to Basic Expert Judgment but with anonymity emphasis):**
- Coordinator presents each expert with a specification and an estimation form.
- Coordinator calls a group meeting where experts discuss estimation issues with the coordinator and each other (this step seems to contradict the "never meet" rule, but the emphasis is on discussion of issues rather than direct negotiation of estimates).
- Experts fill out forms anonymously.
- Coordinator prepares and distributes a summary of the estimation on an iteration form.
- Coordinator calls a group meeting, specifically mentioning noted rationales where estimates varied widely.

- Experts fill out forms anonymously again, and steps 4 and 6 are iterated for as many rounds as appropriate until consensus or stability is achieved.

### Key Takeaways

- [ ] Project estimation is crucial for planning, budgeting, and managing expectations in software development.
- [ ] Estimation methods range from top-down/bottom-up to parametric, expert judgment, analogy, and "price to win."
- [ ] "Price to Win" can secure contracts but often leads to inaccurate cost reflection and potential ethical concerns.
- [ ] Effort and duration are often estimated indirectly through project size, measured in person-months.
- [ ] Source Lines of Code (SLOC) is a conceptually simple size metric but has numerous practical shortcomings in modern development.
- [ ] SLOC struggles with varying coding styles, ignores non-coding activities, correlates poorly with quality, and penalizes high-level languages and code reuse.
- [ ] Expert Judgment-based techniques (Basic, Weighted Average, Consensus, Delphi) leverage human experience for estimation.
- [ ] Analogy estimation relies on comparing new projects to similar past projects, requiring a robust cost database.
- [ ] Weighted Average estimates use optimistic, pessimistic, and most likely values in a formula `(O + 4M + P) / 6` for more robust forecasts.
- [ ] Delphi estimation is a structured, anonymous expert survey in rounds, designed to reduce bias and achieve consensus without direct expert confrontation.

### Conclusion

The array of project estimation techniques discussed in this lecture highlights the multifaceted nature of forecasting software development efforts. From the market-driven "Price to Win" to the experience-based "Expert Judgment" and its structured variations like Delphi, each method offers unique advantages and disadvantages. While metrics like SLOC provide a basic, albeit flawed, measure of size, the trend is towards more sophisticated approaches like Function Points (to be discussed later) that better capture functional complexity. Ultimately, the goal is to move beyond mere guesswork towards a more systematic and data-driven approach to estimation. By understanding these techniques, project managers can select the most appropriate tools for their specific context, leading to more accurate

predictions, better resource allocation, and a higher likelihood of project success in an ever-evolving technological landscape.